

From: Scott Feichter scottfeichter@gmail.com
Subject: Rust
Date: December 20, 2025 at 10:32 PM
To: Jeremy Scott scott@scottjeremy.com

SF

Rust

\$ = a command
< = powershell cmd

rustup = nvm

Linux/Mac
\$ curl --proto '=https' --tlsv1.2 https://sh.rustup.rs -sSf | sh

linker is a program that Rust uses to join its compiled outputs into one file.

C compilers typically include linkers and some common Rust packages depend on C code and will need a C compiler.

Mac:
\$ xcode-select --install

Linux:
Linux users should generally install GCC or Clang

check installation
\$ rustc --version

check path:
\$ echo \$PATH

updating:
\$ rustup update

docs:
rustup doc to open the local documentation in your browser.

IDE support via rust-analyzer

snake_case

main.rs

```
fn main() {  
    println!("Hello, world!");  
}
```

compile:
\$ rustc main.rs

run:
\$./main

The main function is special: It is always the first code that runs in every executable Rust program.

Note: If you want to stick to a standard style across Rust projects, you can use an automatic formatter tool called `rustfmt` to format your code in a particular style (more on `rustfmt` in Appendix D). The Rust team has included this tool with the standard Rust distribution, as `rustc` is, so it should already be installed on your computer!

`println!` calls a Rust macro

`println` calls a function

semicolon (;) which indicates that this expression is over, and the next one is ready to begin. Most lines of Rust code end with a

semicolon (;), which indicates that this expression is over, and the next one is ready to begin. Most lines of Rust code end with a semicolon.

```
$ ls
main main.rs
```

Rust is an *ahead-of-time compiled* language, meaning you can compile a program and give the executable to someone else, and they can run it even without having Rust installed.

Cargo is Rust's build system and package manager.

```
$ cargo --version
```

```
$ cargo new hello_cargo
$ cd hello_cargo
```

Cargo.toml file and a *src* directory with a *main.rs* file inside.

```
.gitignore
```

```
cargo new --help
```

TOML (*Tom's Obvious, Minimal Language*) format, which is Cargo's configuration format.

Cargo expects your source files to live inside the *src* directory. The top-level project directory is just for README files, license information, configuration files, and anything else not related to your code.

One easy way to get that *Cargo.toml* file is to run `cargo init`

```
$ cargo build
```

This command creates an executable file in *target/debug/hello_cargo* (or *target/debug/hello_cargo.exe* on Windows) rather than in your current directory. Because the default build is a debug build, Cargo puts the binary in a directory named *debug*. You can run the executable with this command:

```
$ ./target/debug/hello_cargo # or .\target\debug\hello_cargo.exe on Windows
Hello, world!
```

Running `cargo build` for the first time also causes Cargo to create a new file at the top level: *Cargo.lock*

```
$ cargo run
Finished dev [unoptimized + debuginfo] target(s) in 0.0 secs
Running `target/debug/hello_cargo`
Hello, world!
```

```
cargo check
```

```
cargo new --help
cargo build
cargo run
cargo check
cargo build --release
cargo doc --open
cargo watch -x run
```

This command will create an executable in *target/release* instead of *target/debug*.

```
$ git clone example.org/someproject
$ cd someproject
$ cargo build
```

```
std prelude
```

```
use std::io;
```

```
let mut guess = String::new();
let apples = 5;
```

```
let apples = 5; // immutable
let mut bananas = 5; // mutable
```

`String::new`, a function that returns a new instance of a `String`. `String` is a string type provided by the standard library that is a growable, UTF-8 encoded bit of text.

The `::` syntax in the `::new` line indicates that `new` is an associated function of the `String` type. An *associated function* is a function that's implemented on a type, in this case `String`.

call the `stdin` function from the `io` module, which will allow us to handle user input:

```
io::stdin()
.read_line(&mut guess)
```

```
std::io::stdin
```

The `stdin` function returns an instance of `std::io::Stdin`, which is a type that represents a handle to the standard input for your terminal.

Next, the line `.read_line(&mut guess)` calls the `read_line` method on the standard input handle to get input from the user. We're also passing `&mut guess` as the argument to `read_line` to tell it what string to store the user input in. The full job of `read_line` is to take whatever the user types into standard input and append that into a string (without overwriting its contents), so we therefore pass that string as an argument. The string argument needs to be mutable so that the method can change the string's content.

The `&` indicates that this argument is a *reference*, which gives you a way to let multiple parts of your code access one piece of data without needing to copy that data into memory multiple times

references are immutable by default. Hence, you need to write `&mut guess` rather than `&guess` to make it mutable.

```
.expect("Failed to read line");
```

`read_line` puts whatever the user enters into the string we pass to it, but it also returns a `Result` value. `Result` is an *enumeration*, often called an *enum*, which is a type that can be in one of multiple possible states. We call each possible state a *variant*.

The purpose of these `Result` types is to encode error-handling information. `Result`'s variants are `Ok` and `Err`.

```
let x = 5;
let y = 10;
```

```
println!("x = {x} and y + 2 = {}", y + 2);
```

```
[dependencies]
rand = "0.8.5"
```

SemVer

the *registry*, which is a copy of data from `Crates.io`.

When you *do* want to update a crate, `Cargo` provides the command `update`, which will ignore the `Cargo.lock` file and figure out all the latest versions that fit your specifications in `Cargo.toml`. `Cargo` will then write those versions to the `Cargo.lock` file.

```
use std::io;
```

```
use rand::Rng;
```

```
fn main() {
    println!("Guess the number!");
```

```
let secret_number = rand::thread_rng().gen_range(1..=100);

println!("The secret number is: {secret_number}");

println!("Please input your guess.");

let mut guess = String::new();

io::stdin()
  .read_line(&mut guess)
  .expect("Failed to read line");

println!("You guessed: {guess}");
}
```

The Rng trait defines methods that random number generators implement

A match expression is made up of *arms*.

Scott Feichter
415-702-8512

